

GUIアプリケーション作成手順書

ポリテクセンター栃木

1. 概要

本ドキュメントは、PyQt6フレームワークを使用して構築されたオセロGUIアプリケーションの構造と実装方法を解説する。このアプリケーションは、コアとなるゲームロジック (`othello` パッケージ) とUI層 (`othello_gui` パッケージ) を分離することで、保守性と拡張性の高い設計を実現している。

2. 全体構成とファイルの役割

GUIアプリケーションは、主に以下の3つのファイルで構成されている。

1. `run_gui.py`:

- **役割:** アプリケーションのエントリーポイント（起動スクリプト）。
- **責務:** `QApplication` オブジェクトを生成し、メインウィンドウである `MainWindow` をインスタンス化して表示する。アプリケーション全体の起動と終了を管理する。

2. `othello_gui/main_window.py` (`MainWindow` クラス):

- **役割:** アプリケーションのメインウィンドウであり、UI全体の司令塔。
- **責務:**
 - メニューバー、ステータスラベル、盤面ウィジェット(`BoardWidget`)などのUIコンポーネントをレイアウトする。
 - 「New Game」ダイアログを通じて、ゲームモード（対人/対CPU）や難易度を設定する。
 - `Game` オブジェクトと `Player` オブジェクトを生成・保持し、ゲームの状態を管理する。
 - `BoardWidget` から発信されるシグナル（ユーザーの操作通知）を受け取り、ゲームロジックを呼び出す。
 - ゲームの進行（手番管理、パス、終了判定、結果表示）を制御する。

3. `othello_gui/board_widget.py` (`BoardWidget` クラス):

- **役割:** オセロの盤面を描画し、ユーザーからの入力を受け付ける専用のウィジェット。
- **責務:**
 - `paintEvent` メソッド内で、`Game` オブジェクトの盤面情報(`game.board.grid`)を基に、盤、格子、石を `QPainter` で描画する。
 - `mousePressEvent` メソッド内で、マウスクリックの座標を取得する。
 - クリックされた座標が有効かどうかを検証し、有効であれば `move_made` シグナルを**発信(emit)**して `MainWindow` に通知する。UIの描画と入力受付に特化し、ゲームの複雑な進行ロジックは関知しない。

3. 実装の詳細解説

3.1. エントリーポイント (`run_gui.py`)

PyQtアプリケーションの標準的な起動コード。MainWindowを呼び出すだけのシンプルな構成。

```
import sys
from PyQt6.QtWidgets import QApplication
from othello_gui.main_window import MainWindow

def main():
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())

if __name__ == '__main__':
    main()
```

3.2. 描画の仕組み (BoardWidget)

BoardWidgetのpaintEventが描画処理の核心部分。QPainterを使い、図形を描画していく。

- painter.fillRect(): ウィジェット全体を緑色で塗りつぶす。
- painter.drawLine(): 格子線を引く。
- forループで盤面の全マスをチェックし、game.board.gridの状態に応じてpainter.drawEllipse()で黒または白の円（石）を描画する。

```
# othello_gui/board_widget.py (一部抜粋)
def paintEvent(self, event):
    painter = QPainter(self)
    painter.setRenderHint(QPainter.RenderHint.Antialiasing) # 円を滑らかに描画
    painter.fillRect(self.rect(), QColor("#008000")) # 背景色

    # ...格子線の描画...

    for y in range(grid_size):
        for x in range(grid_size):
            disc = self.game.board.grid[y][x]
            if disc != Disc.EMPTY:
                rect = self.cell_rect(x, y) # 描画範囲を計算
                # ...石の色を設定...
                painter.drawEllipse(rect)
```

3.3. UIとロジックの連携 (シグナルとスロット)

本アプリケーションの設計で最も重要なのが、**シグナルとスロット**によるコンポーネント間の疎結合な連携である。

1. **シグナルの定義 (BoardWidget)**: BoardWidgetは、有効な手が打たれたことを通知するためのシグナル move_madeを定義する。このシグナルは、打たれた手の座標(x, y)をペイロードとして運ぶ。

```
# othello_gui/board_widget.py
class BoardWidget(QWidget):
    move_made = pyqtSignal(int, int) # (x, y)座標を送るシグナル
    # ...
```

2. **シグナルの発信 (BoardWidget)**: `mousePressEvent`内で、クリックされた手が有効手であると確認できたら、`self.move_made.emit(x, y)`を呼び出してシグナルを発信する。

```
# othello_gui/board_widget.py (mousePressEvent内)
if (x, y) in self.game.board.get_valid_moves(...):
    self.move_made.emit(x, y)
```

3. **スロットの定義と接続 (MainWindow)**: `MainWindow`は、シグナルを受け取って処理を実行するメソッド(スロット) `play_turn_and_update`を定義する。そして、`start_new_game`内で、`BoardWidget`の `move_made`シグナルをこのスロットに**接続(connect)**する。

```
# othello_gui/main_window.py
class MainWindow(QMainWindow):
    def start_new_game(self):
        # ...
        # self.board_widget.move_madeシグナルをself.play_turn_and_updateスロットに
        接続
        self.board_widget.move_made.connect(self.play_turn_and_update)
        # ...

    def play_turn_and_update(self, x, y): # スロット
        if self.game_over:
            return
        self.game.play_turn(x, y)
        self.board_widget.update() # 盤面の再描画を指示
        self.handle_next_turn()
```

この仕組みにより、`BoardWidget`は「クリックされた」という事実を通知するだけでよく、その後のゲーム進行を`MainWindow`に委任できる。

3.4. ゲーム進行管理 (MainWindow)

`MainWindow`の`handle_next_turn`メソッドがゲーム進行を管理する。

- プレイヤーを切り替える。
- `game.is_game_over()`でゲーム終了を判定する。
- `game.board.get_valid_moves()`でパスを判定する。
- 現在のプレイヤーがコンピュータ(`ComputerPlayer`)の場合、`QTimer.singleShot()`を使って短い遅延の後、`trigger_computer_move`を呼び出す。これにより、UIがフリーズすることなく、コンピュータが「思考している」ように見せることができる。

```
# othello_gui/main_window.py (一部抜粋)
def handle_next_turn(self):
    # ... プレイヤー切り替え、ゲーム終了判定、パス判定 ...

    if isinstance(self.current_player, ComputerPlayer):
        # 0.5秒後にコンピュータの手を動かす
        QTimer.singleShot(500, self.trigger_computer_move)
```

4. まとめ

このGUIアプリケーションは、以下の優れた設計原則に基づいている。

- **関心の分離**: ゲームロジック、UIの司令塔、盤面の描画という各責務が明確にクラス分割されている。
- **疎結合**: `BoardWidget`と`MainWindow`がシグナルとスロット機構を通じて連携することで、互いの内部実装への依存を最小限に抑えている。
- **応答性**: `QTimer`を使用することで、時間のかかる可能性のある処理（コンピュータの思考）をUIスレッドから分離し、アプリケーションの応答性を維持している。

この構造を理解することで、機能追加（例: ヒント表示、着手履歴）やデザイン変更が容易になる。

Written by ポリテクセンター栃木